Задание 1.

На мой взгляд, поля таблицы более чем корректны, но я бы поменял поле `age`, которое используется для хранения возраста клиента, на более подходящее, что будет хранить дату рождения пользователя `client_birth_date`. Ведь если мы оставим age, которое создаётся в момент добавления пользователя, то оно не изменяется с течением времени, что не логично.

Задание 2.

Необходимо завести отдельную от `items` таблицу `item_prices`, где хранить историю изменения цен.

Убираем из `items` поле со стоимостью:

```
items (
    item_id NUMBER PRIMARY KEY,
    item_name VARCHAR(255)
)
```

Создадим вторую таблицу для учёта цен:

```
item_prices (
    item_id NUMBER FOREIGN KEY REFERENCES
items(item_id),
    price NUMBER,
    valid_from DATE NOT NULL,
    valid_to DATE
)
```

Такая таблица будет хранить историю цен для одного и того же товара, информация о которой может понадобиться для отчётов.

Задание 3.

Исходный запрос содержит несколько проблем:

1. в `GROUP BY` отсутствует `DEPARTMENT_NAME`, группировка же выполняется только по `DEPARTMENT_ID`, в то время как в выборке выводится оба поля: `DEPARTMENT_ID` и `DEPARTMENT_NAME`.
2. некорректная проверка `MANAGER_ID` на равенство `NULL`
3. WHERE не используется для фильтрации групп (`AND SUM(SALARY) >= 100000`)
4. использование неявной связи таблиц через `FROM`, лучше использовать явную связку через `JOIN`

Вариант исправленного запроса:

```
SELECT D.DEPARTMENT_ID,
    D.DEPARTMENT_NAME,
    SUM(E.SALARY) AS TOTAL_SALARY
FROM EMPLOYEES E
JOIN DEPARTMENTS D ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
JOIN LOCATIONS L ON D.LOCATION_ID = L.LOCATION_ID
WHERE E.MANAGER_ID IS NULL
    AND L.CITY = 'SEOUL'
GROUP BY D.DEPARTMENT_ID, D.DEPARTMENT_NAME
HAVING SUM(E.SALARY) > 100000;
```

Задание 4.

Если карандаш упал, то перепрыгнуть его можно только в том случае, если он не укатился в плотную к стене или углу.